
ASSIGNMENT 22

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

<https://github.com/omkulawade03/Frontend-Session-Assignments.git>

Import Required Libraries

```
train_model.py > ...
1  # =====
2  # Import Required Libraries
3  # =====
4
5  import pandas as pd          # For loading and handling dataset
6  import joblib                # For saving and loading ML model
7
8  # Import Machine Learning modules
9  from sklearn.model_selection import train_test_split
10 from sklearn.linear_model import LogisticRegression
11
12 # Import evaluation metrics
13 from sklearn.metrics import (
14     confusion_matrix,
15     accuracy_score,
16     precision_score,
17     recall_score,
18     f1_score,
19     classification_report
20 )
21
```

Q1. (Data Loading & Preprocessing)

Load the Heart Disease dataset.

Separate features into X (independent) and y (dependent/target = HeartDisease).

Perform necessary preprocessing (encoding categorical columns).

Program Code:

#Questions 01

```
22  # =====
23  # Q1. Data Loading & Preprocessing
24  # =====
25
26  # Load Heart Disease Dataset
27  df = pd.read_csv("heart.csv")
28
29  # Display first 5 rows of dataset
30  print("First 5 Records:")
31  print(df.head())
32
33  # Display dataset information
34  print("\nDataset Information:")
35  print(df.info())
36
37  # Display dataset shape
38  print("\nDataset Shape:", df.shape)
39
40  # -----
41  # Separate Features (X) and Target (y)
42  # -----
43
44  # Independent variables (Input Features)
45  X = df.drop("HeartDisease", axis=1)
46
47  # Dependent variable (Target)
48  y = df["HeartDisease"]
49
50  # -----
51  # Encode Categorical Columns
52  # -----
53
54  # Convert categorical columns into numeric format
55  # drop_first=True avoids dummy variable trap
56  X = pd.get_dummies(X, drop_first=True)
57
58  # Save column names for future prediction
59  columns = X.columns
60
61  print("\nEncoded Feature Columns:")
62  print(columns)
63
```

OUTPUT SCREENSHOT:

First 5 Records:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

Dataset Information:

<class 'pandas.DataFrame'>

RangeIndex: 918 entries, 0 to 917

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Age	918 non-null	int64
1	Sex	918 non-null	str
2	ChestPainType	918 non-null	str
3	RestingBP	918 non-null	int64
4	Cholesterol	918 non-null	int64
5	FastingBS	918 non-null	int64
6	RestingECG	918 non-null	str
7	MaxHR	918 non-null	int64
8	ExerciseAngina	918 non-null	str
9	Oldpeak	918 non-null	float64
10	ST_Slope	918 non-null	str
11	HeartDisease	918 non-null	int64

dtypes: float64(1), int64(6), str(5)

memory usage: 97.6 KB

None

Dataset Shape: (918, 12)

Encoded Feature Columns:

```
Index(['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak',  
      'Sex_M', 'ChestPainType_ATA', 'ChestPainType_NAP', 'ChestPainType_TA',  
      'RestingECG_Normal', 'RestingECG_ST', 'ExerciseAngina_Y',  
      'ST_Slope_Flat', 'ST_Slope_Up'],  
      dtype='str')
```

Q2. (Train-Test Split)

Split the preprocessed data into training and testing sets using `train_test_split`.

Use `test_size=0.20` and `random_state=42`.

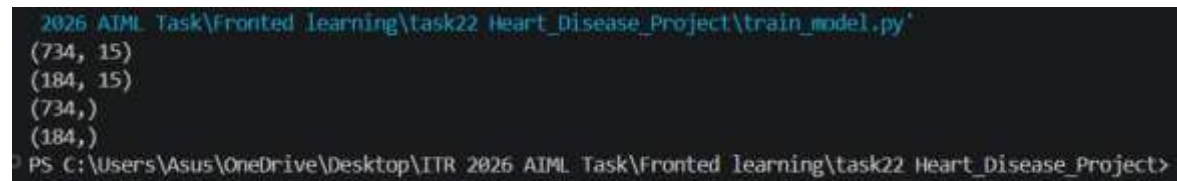
Print the shape of `X_train`, `X_test`, `y_train`, and `y_test`.

Program Code:

#Questions 02

```
64 # =====
65 # Q2. Train-Test Split
66 # =====
67 import pandas as pd
68 from sklearn.model_selection import train_test_split
69
70 # Load dataset
71 df = pd.read_csv("heart.csv")
72
73 # Separate Features and Target
74 X = df.drop("HeartDisease", axis=1)
75 y = df["HeartDisease"]
76
77 # Encode categorical columns
78 X = pd.get_dummies(X, drop_first=True)
79
80 # Now Split the Data
81 X_train, X_test, y_train, y_test = train_test_split(
82     X,
83     y,
84     test_size=0.20,
85     random_state=42
86 )
87
88 print(X_train.shape)
89 print(X_test.shape)
90 print(y_train.shape)
91 print(y_test.shape)
92
```

OUTPUT SCREENSHOT:



```
2026 AIIML Task\Fronted learning\task22 Heart_Disease_Project\train_model.py`  
(734, 15)  
(184, 15)  
(734, )  
(184, )  
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIIML Task\Fronted learning\task22 Heart_Disease_Project>
```

Q3. (Building Logistic Regression Model)

Train a Logistic Regression model:

Import LogisticRegression from sklearn.

Create the model and fit it on the training data (X_train, y_train).

Program Code:

#Questions 03

```
93  # =====
94  # Q3. Building Logistic Regression Model
95  # =====
96
97  # Import Logistic Regression algorithm
98  from sklearn.linear_model import LogisticRegression
99
100 # -----
101 # Create the Logistic Regression model
102 # -----
103
104 # max_iter=1000 increases the maximum number
105 # of iterations to ensure the model converges.
106 model = LogisticRegression(max_iter=1000)
107
108 # -----
109 # Train (Fit) the Model
110 # -----
111
112 # Train the model using the training dataset
113 model.fit(x_train, y_train)
114
115 # Display success message
116 print("\nLogistic Regression Model Trained Successfully!")
117
```

OUTPUT SCREENSHOT:

```
Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
  https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
  https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(

Logistic Regression Model Trained Successfully!
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task22 Heart_Disease_Project>
```

Q4. (Making Predictions)

Use the trained model to predict on the test data.

Store predictions in `y_pred`.

Print the first 10 actual values (`y_test`) and predicted values (`y_pred`).

Program Code:

#Questions 04

```
118 # =====
119 # Q4. Making Predictions
120 # =====
121
122 # Predict values using testing dataset
123 y_pred = model.predict(x_test)
124
125 # Display first 10 actual values
126 print("\nFirst 10 Actual Values:")
127 print(y_test.values[:10])
128
129 # Display first 10 predicted values
130 print("\nFirst 10 Predicted Values:")
131 print(y_pred[:10])
132
```

OUTPUT SCREENSHOT:

```
First 10 Actual Values:
```

```
[0 1 1 1 0 1 1 0 1 1]
```

```
First 10 Predicted Values:
```

```
[0 0 1 1 0 1 1 0 1 1]
```

```
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task22 Heart_Disease_Project>
```

Q5. (Confusion Matrix)

Create a Confusion Matrix using `confusion_matrix(y_test, y_pred)`.

Display the matrix.

Label TN, FP, FN, and TP in the output.

Program Code:

#Questions 05

```
134 # =====
135 # Q5. Confusion Matrix
136 # =====
137
138 # Generate Confusion Matrix
139 cm = confusion_matrix(y_test, y_pred)
140
141 print("\nConfusion Matrix")
142 print(cm)
143
144 # Extract values from Confusion Matrix
145 TN = cm[0][0]
146 FP = cm[0][1]
147 FN = cm[1][0]
148 TP = cm[1][1]
149
150 # Display Confusion Matrix labels
151 print("\nTrue Negative (TN):", TN)
152 print("False Positive (FP):", FP)
153 print("False Negative (FN):", FN)
154 print("True Positive (TP):", TP)
155
```

OUTPUT SCREENSHOT:

```
Confusion Matrix
[[67 10]
 [17 90]]

True Negative (TN): 67
False Positive (FP): 10
False Negative (FN): 17
True Positive (TP): 90
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AI ML Task\Fronted learning\task22 Heart_Disease_Project>
```

Q6. (Model Evaluation Metrics)

Calculate the following metrics:

Accuracy

Precision

Recall

F1 Score

Use `classification_report(y_test, y_pred)` and print the full report.

Program Code:

#Questions 06

```
156 # =====
157 # Q6. Model Evaluation
158 # =====
159
160 # Calculate Accuracy
161 accuracy = accuracy_score(y_test, y_pred)
162
163 # Calculate Precision
164 precision = precision_score(y_test, y_pred)
165
166 # Calculate Recall
167 recall = recall_score(y_test, y_pred)
168
169 # Calculate F1 Score
170 f1 = f1_score(y_test, y_pred)
171
172 # Display Evaluation Metrics
173 print("\nModel Evaluation")
174
175 print("Accuracy :", accuracy)
176 print("Precision:", precision)
177 print("Recall   :", recall)
178 print("F1 Score :", f1)
179
180 # Display Classification Report
181 print("\nClassification Report")
182 print(classification_report(y_test, y_pred))
183
```

OUTPUT SCREENSHOT:

```
Model Evaluation
Accuracy : 0.8532608695652174
Precision: 0.9
Recall   : 0.8411214953271028
F1 Score : 0.8695652173913043

Classification Report
              precision    recall  f1-score   support

     0           0.80       0.87       0.83         77
     1           0.90       0.84       0.87        107

 accuracy              0.85              184
  macro avg           0.85       0.86       0.85              184
 weighted avg           0.86       0.85       0.85              184
```

Q7. (Saving the Model)

Save the trained model and preprocessing objects using joblib:

Model → "heart_model.pkl"

Scaler (if used) → "scaler.pkl"

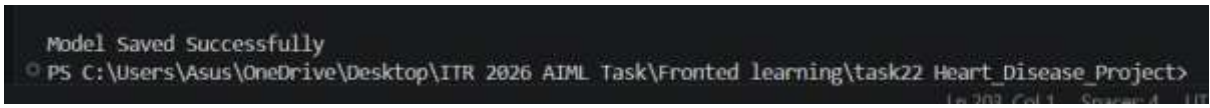
Column list → "columns.pkl"

Program Code:

#Questions 07

```
184 # =====
185 # Q7. Save Model
186 # =====
187
188 # Save trained Logistic Regression model
189 joblib.dump(model, "heart_model.pkl")
190
191 # Save feature column names
192 joblib.dump(columns, "columns.pkl")
193
194 # Save scaler
195 # (No scaler used in this project, so saving None)
196 joblib.dump(None, "scaler.pkl")
197
198 print("\nModel Saved Successfully")
199
```

OUTPUT SCREENSHOT:



```
Model Saved Successfully
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task22 Heart_Disease_Project>
```

The screenshot shows a terminal window with a dark background. The first line displays the message "Model Saved Successfully". The second line shows a PowerShell prompt "PS" followed by the file path "C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task22 Heart_Disease_Project>".

Q8. (Loading & Testing Saved Model)

Load the saved model and objects.

Create sample input data (one patient record).

Preprocess the input (encoding + scaling).

Make a prediction using the loaded model and print the result.

Program Code:

#Questions 08

```
201  # =====
202  # Q8. Loading & Testing Saved Model
203  # =====
204
205  # Import required libraries
206  import pandas as pd
207  import joblib
208
209  # -----
210  # Load Saved Model and Objects
211  # -----
212
213  # Load the trained model
214  loaded_model = joblib.load("heart_model.pkl")
215
216  # Load the feature column names
217  loaded_columns = joblib.load("columns.pkl")
218
219  # Load the scaler (None in this project)
220  loaded_scaler = joblib.load("scaler.pkl")
221
222  print("Model Loaded Successfully!")
223
```

```
224 # -----
225 # Create Sample Input Data
226 # -----
227
228 sample = {
229     "Age": 45,
230     "Sex": "M",
231     "ChestPainType": "ATA",
232     "RestingBP": 130,
233     "Cholesterol": 230,
234     "FastingBS": 0,
235     "RestingECG": "Normal",
236     "MaxHR": 150,
237     "ExerciseAngina": "N",
238     "Oldpeak": 1.2,
239     "ST_Slope": "Up"
240 }
241
242 # Convert dictionary to DataFrame
243 sample_df = pd.DataFrame([sample])
244
```

```
245 # -----
246 # Preprocess the Input Data
247 # -----
248
249 # Apply One-Hot Encoding
250 sample_df = pd.get_dummies(sample_df)
251
252 # Match feature columns with training data
253 sample_df = sample_df.reindex(columns=loaded_columns, fill_value=0)
254
255 # Apply scaling if a scaler exists
256 if loaded_scaler is not None:
257     sample_df = loaded_scaler.transform(sample_df)
258
259 # -----
260 # Make Prediction
261 # -----
262
263 prediction = loaded_model.predict(sample_df)
264
265 # Display Prediction Result
266 print("\nPrediction Result:")
267
268 if prediction[0] == 1:
269     print("💖 Heart Disease: YES")
270 else:
271     print("💚 Heart Disease: NO")
272
```

OUTPUT SCREENSHOT:

```
Model Loaded Successfully!
```

```
Prediction Result:
```

```
♥ Heart Disease: NO
```

```
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted lea
```

Program Code:

#Questions 09

```
app.py > ...
1  # =====
2  # Q9. Streamlit Input Interface
3  # =====
4
5  import streamlit as st
6
7  # Page Configuration
8  st.set_page_config(page_title="Heart Disease Prediction")
9
10 # Title
11 st.title("❤️ Heart Disease Prediction System")
12
13 st.write("Enter the patient details below.")
14
15 # -----
16 # Input Fields
17 # -----
18
19 age = st.number_input("Age", min_value=1, max_value=120, value=45)
20
21 sex = st.selectbox("Sex", ["M", "F"])
22
23 chest_pain = st.selectbox(
24     "Chest Pain Type",
25     ["ATA", "NAP", "ASY", "TA"]
26 )
27
28 resting_bp = st.number_input(
29     "Resting Blood Pressure",
30     value=120
31 )
32
33 cholesterol = st.number_input(
34     "Cholesterol",
35     value=200
36 )
```

```
app.py > ...
37
38     fasting_bs = st.selectbox(
39         "Fasting Blood Sugar",
40         [0, 1]
41     )
42
43     resting_ecg = st.selectbox(
44         "Resting ECG",
45         ["Normal", "ST", "LVH"]
46     )
47
48     max_hr = st.number_input(
49         "Maximum Heart Rate",
50         value=150
51     )
52
53     exercise_angina = st.selectbox(
54         "Exercise Induced Angina",
55         ["Y", "N"]
56     )
57
58     oldpeak = st.number_input(
59         "Oldpeak",
60         value=1.0
61     )
62
63     st_slope = st.selectbox(
64         "ST Slope",
65         ["Up", "Flat", "Down"]
66     )
67
68     # -----
69     # Predict Button
70     # -----
71
72     if st.button("Predict"):
73         st.success("Input received successfully!")
```

OUTPUT SCREENSHOT:



Heart Disease Prediction System

Enter the patient details below.

Age

45

-

+

Sex

M

▼

Chest Pain Type

NAP

▼

Resting Blood Pressure

120

-

+

Cholesterol

200

-

+

Fasting Blood Sugar

1

▼

Resting ECG

ST

▼

Maximum Heart Rate

150

-

+

Exercise Induced Angina

Y

▼

Oldpeak

1.00

-

+

ST Slope

Up

▼

Predict

Input received successfully!

Q10. (Complete Streamlit Web App)

Complete the Streamlit app for Heart Disease Prediction:

Load the saved model and preprocessing objects.

On button click, preprocess input and show prediction ("Heart Disease: Yes"

or "No") with `st.success()` or `st.error()`.

Program Code:

#Questions 10

Q10app.py > ...

```
1  # =====
2  # Q10. Complete Streamlit Web App
3  # =====
4
5  # Import Required Libraries
6  import streamlit as st
7  import pandas as pd
8  import joblib
9
10 # -----
11 # Load Saved Model and Objects
12 # -----
13
14 model = joblib.load("heart_model.pkl")
15 columns = joblib.load("columns.pkl")
16 scaler = joblib.load("scaler.pkl")
17
18 # -----
19 # Streamlit Page
20 # -----
21
22 st.set_page_config(page_title="Heart Disease Prediction")
23
24 st.title("❤️ Heart Disease Prediction System")
25 st.write("Enter the patient details below.")
26
27 # -----
28 # Input Fields
29 # -----
30
31 age = st.number_input("Age", min_value=1, max_value=120, value=45)
32
33 sex = st.selectbox(
34     "Sex",
35     ["M", "F"]
36 )
```

Q10app.py > ...

```
37
38 chest_pain = st.selectbox(
39     "Chest Pain Type",
40     ["ATA", "NAP", "ASY", "TA"]
41 )
42
43 resting_bp = st.number_input(
44     "Resting Blood Pressure",
45     value=120
46 )
47
48 cholesterol = st.number_input(
49     "Cholesterol",
50     value=200
51 )
52
53 fasting_bs = st.selectbox(
54     "Fasting Blood Sugar",
55     [0, 1]
56 )
57
58 resting_ecg = st.selectbox(
59     "Resting ECG",
60     ["Normal", "ST", "LVH"]
61 )
62
63 max_hr = st.number_input(
64     "Maximum Heart Rate",
65     value=150
66 )
67
68 exercise_angina = st.selectbox(
69     "Exercise Induced Angina",
70     ["Y", "N"]
71 )
72
```

Q10app.py > ...

```
72
73 oldpeak = st.number_input(
74     "Oldpeak",
75     value=1.0
76 )
77
78 st_slope = st.selectbox(
79     "ST Slope",
80     ["Up", "Flat", "Down"]
81 )
82
83 # -----
84 # Predict Button
85 # -----
86
87 if st.button("Predict"):
88     # Create patient record
89     sample = pd.DataFrame([
90         "Age": age,
91         "Sex": sex,
92         "ChestPainType": chest_pain,
93         "RestingBP": resting_bp,
94         "Cholesterol": cholesterol,
95         "FastingBS": fasting_bs,
96         "RestingECG": resting_ecg,
97         "MaxHR": max_hr,
98         "ExerciseAngina": exercise_angina,
99         "Oldpeak": oldpeak,
100         "ST_Slope": st_slope
101     ])
102
103 # One-Hot Encoding
104 sample = pd.get_dummies(sample)
105
106 # Match columns with training data
107 sample = sample.reindex(columns=columns, fill_value=0)
```

```
109
110     # Apply scaler (if used)
111     if scaler is not None:
112         sample = scaler.transform(sample)
113
114     # Predict
115     prediction = model.predict(sample)
116
117     # Display Result
118     if prediction[0] == 1:
119         st.error("❤️ Heart Disease: Yes")
120     else:
121         st.success("💚 Heart Disease: No")
122
```

OUTPUT SCREENSHOT:



Heart Disease Prediction System

Enter the patient details below.

Age

45

-

+

Sex

M

▼

Chest Pain Type

ATA

▼

Resting Blood Pressure

120

-

+

Cholesterol

200

-

+

Fasting Blood Sugar

0

▼

Resting ECG

Normal

▼

Maximum Heart Rate

150

-

+

Exercise Induced Angina

Y

▼

Oldpeak

1.00

-

+

ST Slope

Up

▼

Predict

 Heart Disease: No

